VDK/TLM Creation

Rapid Training

X-2025.06

Module 1: Lab B

Test Driven Design with TLM Creator

© 2025 Synopsys, Inc.

Table of Contents

Lab Overview *2*

Task-1: Open the Timer Project **2**

Task-2: Add New Model Unit Tests **4**

Task-3: Update Model Design **6**

Task-4: Functional Coverage **9**

Task-5: Add Model Behavior (TDD) **12**

Task-6: Adding a Unit test for the Set-and-forget behavior (Optional task) **16**

Task-7: What's Next **19**

Lab Overview

Your task for this lab is to use the TLM Creation perspective of Virtualizer Studio to test and implement some additional behavior for the timer model that you created in Module 1 Lab A.

We will be using the Test-Driven Methodology as explained in the VDK Creation Methodology. For details, refer to the *TLM Creation Training Lecture – Introduction*.

In the pre-implementation phase of the last exercise (Lab A), you had created / reviewed an Excel spreadsheet `TimerFunctionalityTracking.xlsx` listing the component's features for tracking. You can use that copy for performing this step.

Steps to be followed:

1. You need to go over the list of features, one-by-one, write a unit test for it, and implement the behavior. Save the changes, re-build, re-run unit tests, check for failures, fix the code, save, re-run the unit test, all until the new tests are passing.
2. After the test pass, mark them in the tracking spreadsheet as *implemented and tested*. The order of the implemented features and unit tests is up to you, based on your skills, knowledge, and preferences.

Task-1: Open the Timer Project

***Motivation**: We had already created the shell of the timer model in Lab A. Now, we will add some additional behavior to the model implementation and add required tests to validate them.*

If you have completed the previous lab (Module 1 Lab A), you can just open Virtualizer Studio and continue with the same `timer` TLMC project and skip this task. Continue with Task-2.

1. If have not completed the previous lab or want to start from a prepared starting point for this lab, use the following steps to create the starting point.

   i. Open a console and invoke Virtualizer studio from the Synopsys Virtual Prototyping installation.

   ```
   % source <installDir>/SLS/windows/setup.bat vi[rtualizer] (Windows)

   % source <installDir>/SLS/linux/setup.csh|sh vi[rtualizer] (Linux)
   ```

   This sourcing option picks up the Virtualizer *Elite* or *Base* license in that order, whichever is available. For more details on other options, refer to the VP documentation.

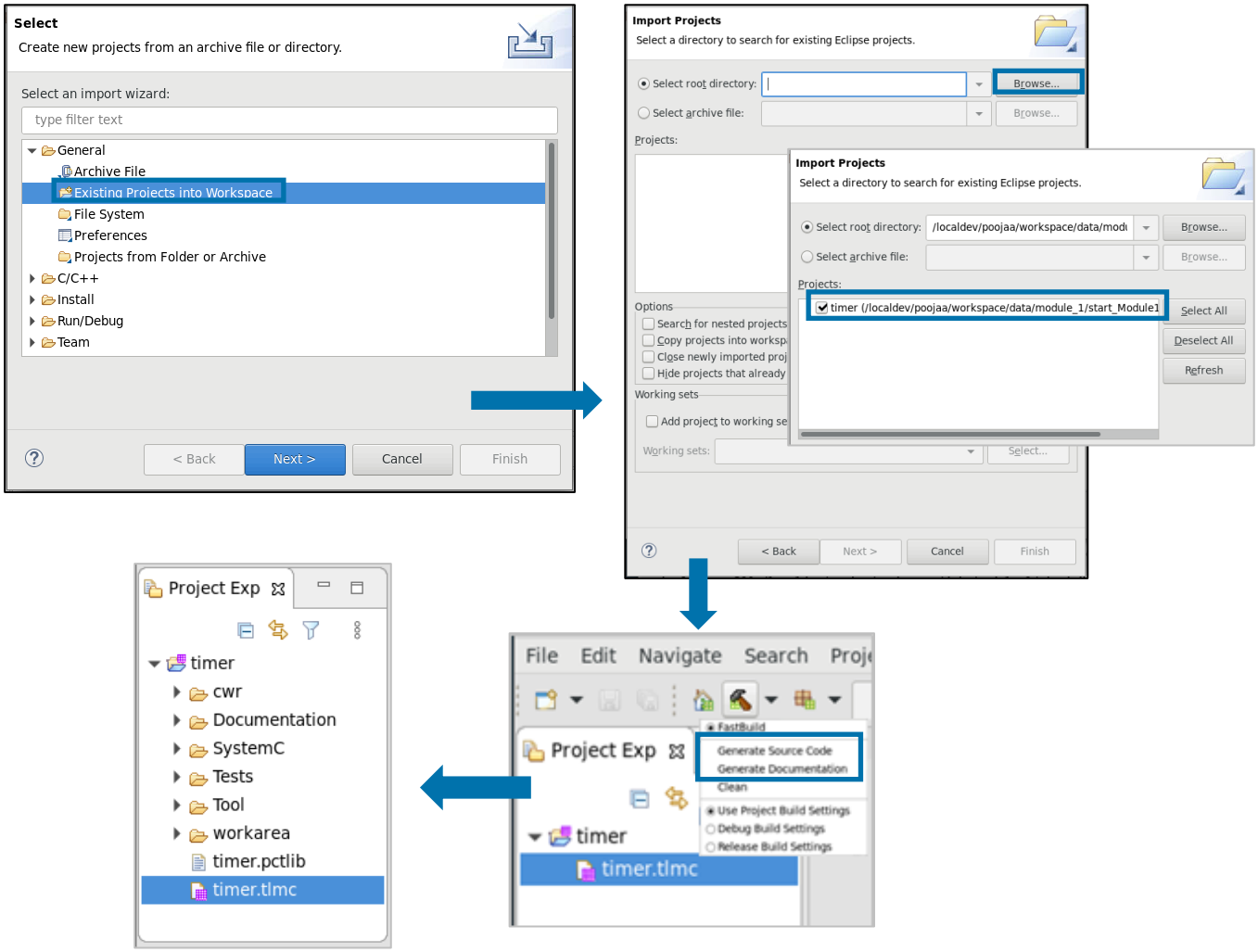
   ii. Go to the `<Lab-data>/data/module_1` directory and start Virtualizer Studio.

   ```
   vs -d start_Module1LabB &
   ```

2. Close the welcome screen and import the starting point TLM Creator Project by clicking ***File*** → ***Import…*** menu option.

   i. In the ***Import*** dialog, select ***General*** → ***Existing Projects into Workspace*** to select what you want to import.

   ii. Click ***Next*** to get to the ***Import Projects*** dialog. Browse to the `timer` directory in the `start_Module1LabB` directory and click ***Finish*** (Make sure you are in the right directory – it's easy to end up in the wrong directory!).



iii. Select the `timer.tlmc` file in the Project Explorer and click ***Generate Sources Code*** from the ***Build*** toolbar button options. Also, generate the documentation using the ***Generate Documentation*** option.

This will provide you with the starting point `timer` model for this lab.

Task-2: Add New Model Unit Tests

***Motivation**: In Lab A, we have seen that TLM Creator provides a set of default unit tests as well as behavior implementations in our TLM project. Now, we will try to add more unit tests and behavior as per our model specifications.*

We have already introduced you to the `timerTest.cc` file, its initial content and the unit test basic infrastructure. Note, that this file does not get regenerated by Virtualizer Studio on the model save/build. Therefore, your changes are retained. This file contains class constructor, destructor, `initialize` method, `setup` method and default unit tests. All these methods contain initial code that can be used as a starting point for your implementation. Let's now add a new test to this file.

Adding new Unit test

Add a unit test, `RegisterReset_Test1` to verify if all the registers are properly reset.

3. Open the TimerFuntionalityTracking spreadsheet, review the requirements on reset behavior, and identify the items that you are going to test, as shown on the right-hand slide.

ITCSR					**9.6.5 ITCSR**
ITCSR		**1**			32 bit register
ITCSR		**2**			masking read/write lower 8bit read write
ITCSR		**3**			masking: upper 24 bit always read 0
ITCSR		**4**		x	reset: lower 0-bit as 0 (0x00)
ITCSR		**5**			freeze/unfreeze timer (STOP, DOZE, DGB bit)
ITCSR		**6**			direct overvwrite of counter value only if OVW is 1
ITCSR		**7**			ITIF cleared when writing 1 to ITIF bit
ITCSR		**8**			ITIF cleared when writing to ITDR
ITCSR		**9**			ITIF shows interrupt present
ITCSR		**10**			PIT interrupt is present
ITCSR		**11**			reload from latch
ITCSR		**12**			reload from 0xFFFF
ITCSR		**13**			enable the PIT
ITDR					**9.6.6 ITDR**
ITDR		**1**			write the timer modulus
ITDR		**2**			value is used during timer reload
ITDR		**3**			read return the modulus
ITDR		**4**			write can also change counter if OVW is 1
ITDR		**5**			counter is read from the ITADR register
ITDR		**6**			32 bit access only
ITDR		**7**			masking : upper 16 bit always read as 0
ITDR		**8**		x	reset: lower 16-bit as 1 (0xFFFF)
ITADR					**9.6.7 ITADR**
ITADR		**1**			read only register
ITADR		**2**			32 bit access only
ITADR		**3**			read return the counter value (16 bit)
ITADR		**4**			masking: upper 16bit are always read as zero
ITADR		**5**		x	reset: lower 16-bit as 1 (0xFFFF)

4. To implement these tests, double-click on the file `timerTest.cc` in ***Project Explorer*** to open it for editing and perform the following steps.

   i. Add the following test definition between the `SCML2_BEGIN_TESTS` and `SCML2_END_TESTS` macros to register your test:

```
SCML2_TEST(RegisterReset_Test1);
```

   ii. Add the following implementation code for the `RegisterReset_Test1` method after the default `testAlwaysSucceeds` function:

```
void RegisterReset_Test1(){
        SCML2_INFO_TO(this->mirrorModel,FUNCTIONAL_LOG) <<
          "### ITSCR.3+4 (upper bits always 0, lower 8 bits 0x00 on reset)" << std::endl;
        SCML2_ASSERT_THAT(this->p_registers.read32(ITCSR).equals(0));

        SCML2_INFO_TO(this->mirrorModel,FUNCTIONAL_LOG) <<
          "### ITDR.7+8 (upper bits always 0, lower 16 bits 0xFFFF on reset)" << std::endl;
        SCML2_ASSERT_THAT(this->p_registers.read32(ITDR).equals(0xFFFF));

        SCML2_INFO_TO(this->mirrorModel,FUNCTIONAL_LOG) <<
          "### ITADR.4+5 (upper bits always 0, lower 16 bits 0xFFFF on reset)" << std::endl;
        SCML2_ASSERT_THAT(this->p_registers.read32(ITADR).equals(0xFFFF));
}
```

iii. Next, we need to enable logging and set the model internal level in the `initialize` function. Also, set the clock period from the test bench.

```
virtual void initialize() {
        // enable all FastTrack messages:
        enable_logging();
        // set MODEL_INTERNAL log level to 4:
        this->modelUnderTest->get_tagged_message_handler()->set_model_internal_log_level(4);
        // get used clock period from test bench:
        m_p_clk_Period = this->mirrorModel->p_clk_mockup.m_period;
}
// Data Members
sc_core::sc_time m_p_clk_Period;
```

iv. Next, add the following code to the `setup` function.

```
virtual void setup() {
        // put all pin into default position, do reset
        coverModel->set_enabled(true);
        wait(2 * this->m_p_clk_Period);
        this->p_reset.set_duration(8*this->m_p_clk_Period);
        this->p_reset.do_reset_pulse();
        wait(2 * this->m_p_clk_Period);
}
```

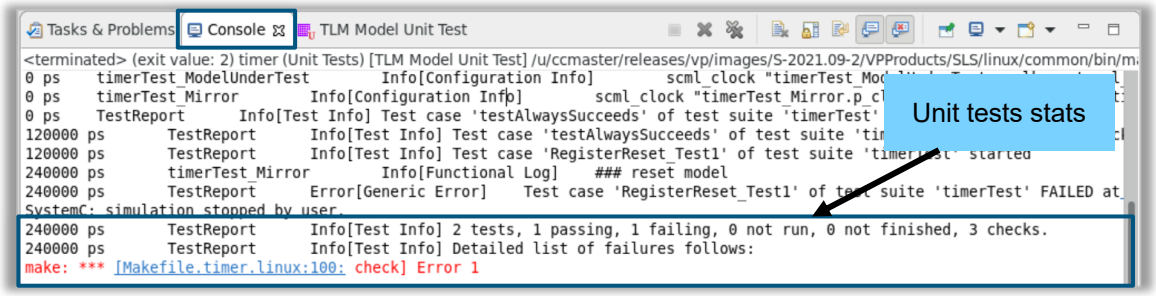
v. Save your edits, e.g. using the ***Save / Save All*** buttons ( ) from the top-level menu. The unit test is now ready to be built and run.

5. Run the unit tests by clicking the arrow next to the ***Play*** toolbar button ( ) and select the ***Run As > TLM Model Unit Test*** option.

Review UnitTest Output

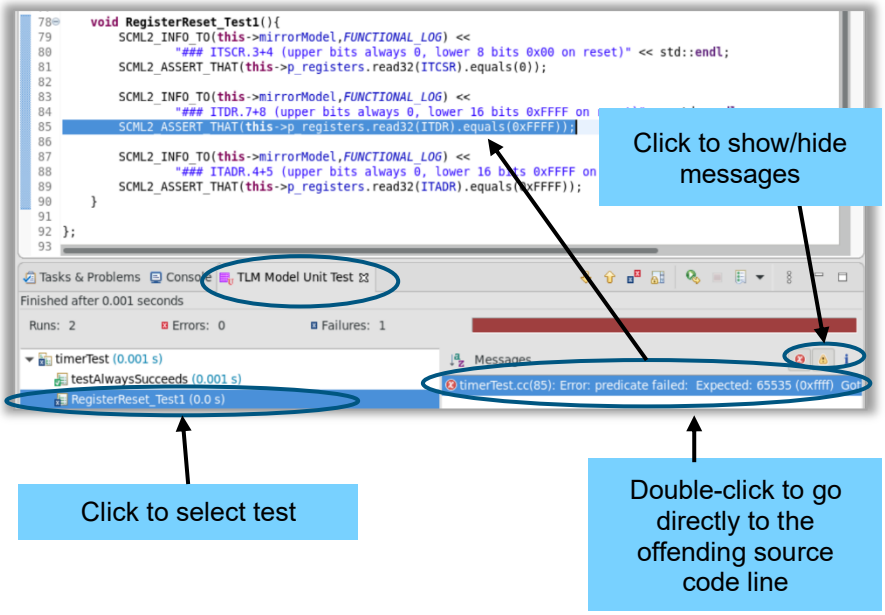
You can review the reported messages from the model build and tests output in the following views:

- ***Console*** view: The build messages and unit tests results are displayed in the ***Console***, which are helpful in debugging. It also displays the unit test statistics as shown in the following illustration.



- The ***TLM Model Unit Test*** view shows the report on the number of tests, errors and failures and allows easy navigation.

- Click on the failed test in the left of the window to get list of errors messages on the right side. Double-click on the error message to open the target source code location of the failed test assertion.



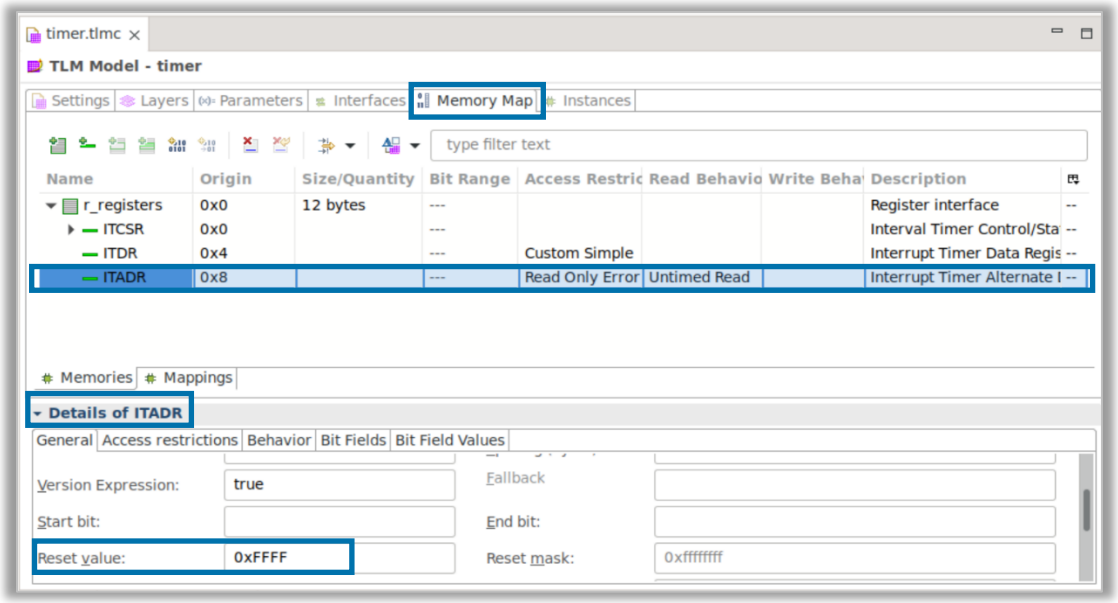
From the output, you can see that the reset test is failing, which is no wonder as per TDD guideline the test was implemented before implementing the functionality.

Task-3: Update Model Design

Motivation: *Let's now implement the missing reset behavior we have identified in the last section. For the missing implementation there are 2 aspects. First you have to specify the reset values of the different registers, and next you need to make sure these values are applied when the reset signal is active.*

6. Click `timer.tlmc` tab to open the model configuration settings in the ***Model Editor*** window.

7. Open the ***Memory Map*** tab and for both, `ITDR` and `ITADR` registers, set their ***Reset Value*** to `0xFFFF`. *For ITCSR, set the Reset Value as 0x0.*



8. Save your project and build the model. Review the messages and re-open the `timerBase.h` file in the editor to verify if the `reset_model ()` method has been updated as follows.

```
, r_registers("r_registers", (12) / scml2::sizeOf<unsigned int >())
, ITCSR(sc_core::sc_gen_unique_name("ITCSR", true), r_registers, (0x0) / scml2::sizeOf<unsigned int >())
, ITCSR_reset_value(0x0)
, ITDR(sc_core::sc_gen_unique_name("ITDR", true), r_registers, (0x4) / scml2::sizeOf<unsigned int >())
, ITDR_reset_value(0xFFFF)
, ITADR(sc_core::sc_gen_unique_name("ITADR", true), r_registers, (0x8) / scml2::sizeOf<unsigned int >())
, ITADR_reset_value(0xFFFF) {
```

```
virtual void reset_model() {
  reset_model_if_available(p_registers_router);
  ITCSR = ITCSR_reset_value;
  ITDR = ITDR_reset_value;
  ITADR = ITADR_reset_value;
}
```

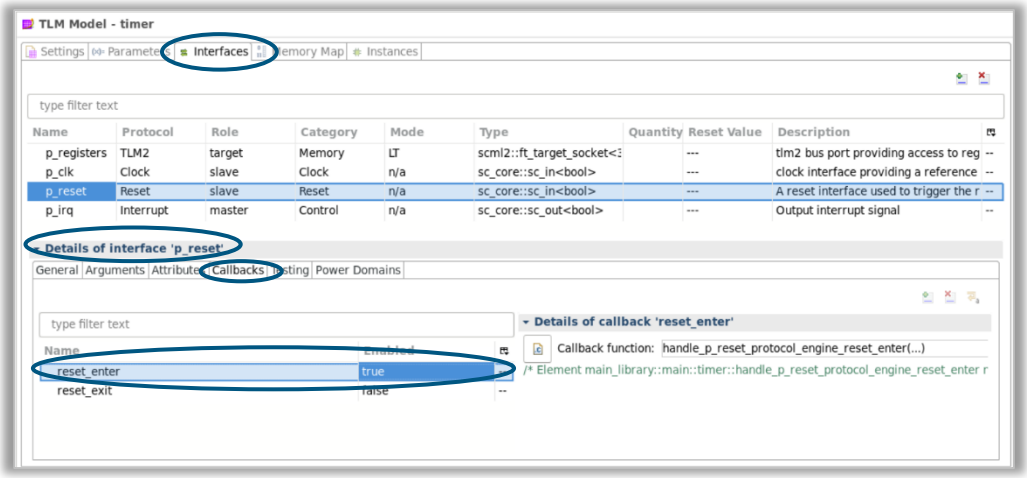
Review the setting of the reset values of the registers in the Constructor and the *reset_model()* definition in the generated timerBase.h file

9. Now that you have defined the register reset values you need to make sure they are also applied when the reset signal is active. Open the ***Interfaces*** tab of the `timer.tlmc` project, click to select the *p_reset* port, and

i. Expand the ***Details of interface 'p_reset'*** pane

ii. Select ***Callbacks*** tab. Note that the `reset_enter` callback is not enabled. Enable it by selecting `true` in the ***Enabled*** column.

iii. Save your Project. Note: Saving the project is important to generate the default callback code.



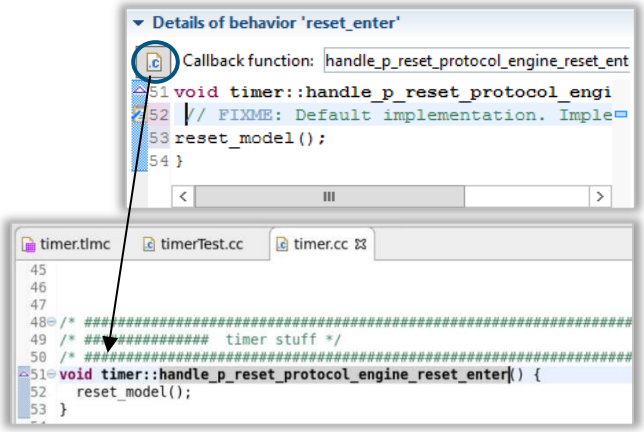
iv. Adapt the callbacks code in the peep-hole: Insert the following code.

```
reset_model();
```

and remove the `//FIXME` … comment of the default implementation.

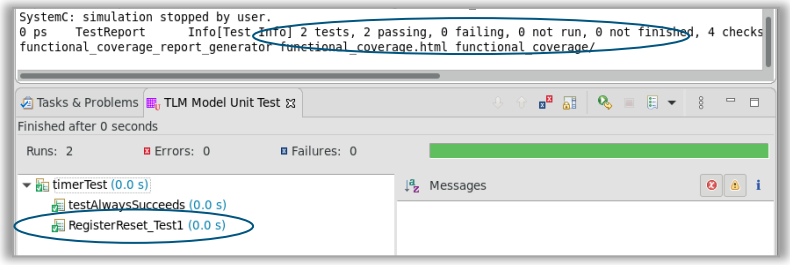
You can also edit or review the code in `timer.cc` file by using the button in the pane.

v. Save your project again using the ***Save*** button on the toolbar.



10. Re-run the unit test and verify the results.

The `RegisterReset_Test1` test is now passing.



Next, we can verify how well the unit tests cover the generated code. One helpful tool for this is the built-in functional coverage.

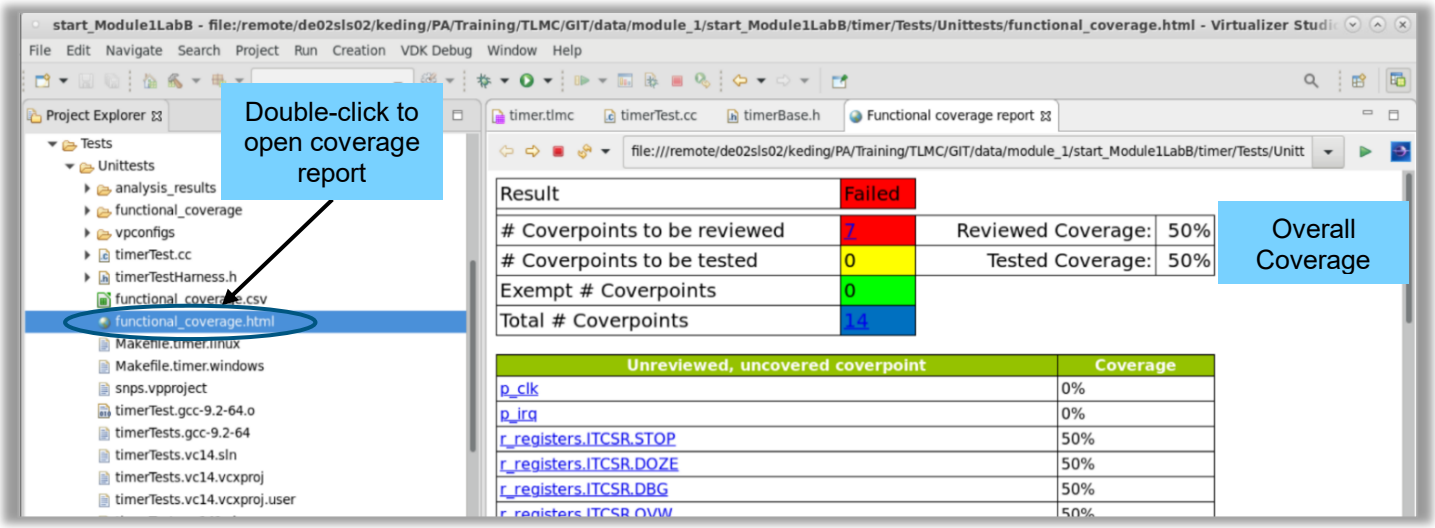
Task-4: Functional Coverage

Motivation: *To validate the quality and completeness of the tests written for a model, TLM Creator provides automated functional coverage. The functional coverage gives you good overview what functionality is covered and what functionality still needs to be tested. The goal of the functional coverage is to provide a measure determining how well the tests cover all the features of a TLM component.*
Here in this task, we will use the TLM Creator Coverage Reports to determine what is already tested, what is left to be tested, and which aspects can be omitted, e.g. as they are not required for the overall project.

The functional coverage is automatically enabled in the TLM Creation perspective of the Virtualizer Studio. By running the unit tests, the functional coverage report is also generated, and you can view or check which parts of the model, that is interfaces, memories, registers, bit-fields, parameters, etc. are tested and which are not.

11. To view the coverage results, expand `timer/Tests/Unittests` in ***Project Explorer***, and double-click on the `functional_coverage.html` file.

Note: By default, the HTML document is opened in the host system's default browser rather than in the Virtualizer Studio environment. You can change this behavior in your environment using the ***Web Browser*** settings in the Preferences dialog. For more details, refer to the Module 1 Lab A.



The functional coverage HTML report provides an overview of the coverage of a component during the unit tests. The report includes the following details for the timer model.

12. As you see the overall coverage number for the component (50%) as the reset test is already exercising a part of the interfaces and registers. On the top you see the overall result (Failed) and next to that the number of *Coverpoints to be reviewed* (7). The overall result will stay *Failed* until the *Coverpoints to be reviewed* is not 0, so the first goal is to bring this number down as much as possible.

    Hence, click on the number 7 (*Coverpoints to be reviewed*) to get to the list of unreviewed coverpoints.

13. In the list of unreviewed and uncovered coverpoints you find the clock and interrupt ports, but also all the bitfields of the ITCSR register. According to the test plan (see TimerFunctionalityTracking.xlsx spreadsheet) some of the bitfields are in the RTL spec, but are not required for the Virtual Prototype project:

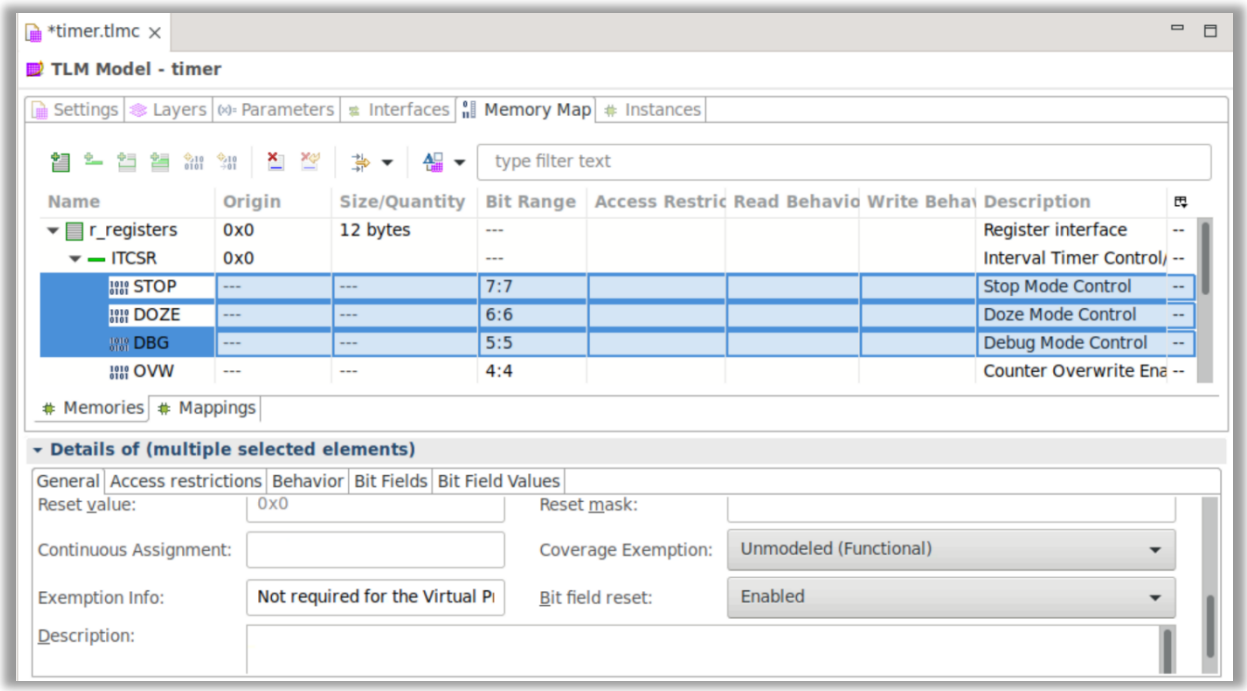
ITCSR					9.6.5 ITCSR	
ITCSR		1			32 bit register	
ITCSR		2			masking read/write lower 8bit read write	
ITCSR		3			masking: upper 24 bit always read 0	
ITCSR		4		x	reset: lower 0-bit as 0 (0x00)	
ITCSR		5		-	freeze/unfreeze timer (STOP, DOZE, DGB bit)	Not required for the Virtual Prototype project
ITCSR		6			direct overvwrite of counter value only if OVW is 1	

14. Hence you can mark the item with a dash in the spreadsheet and exempt them as *Unmodeled (Functional)* in the TLM Creator project.
    The other bitfields of the *ITCSR* register, as well as the *ITDR* and *ITADR* register need to be tested, so exempt them as *Untested (Unsafe)*. This indicates you have reviewed the coverpoints and confirmed they need to be covered by tests.

    i. In the ***Memory Map*** tab of the timer project, select the *STOP*, *DOZE*, and *DBG* bitfields of the *ITCSR* register

    ii. In the Details/General section, select *Unmodeled (Functional)* as ***Coverage Exemption*** for these bitfields.

    iii. Enter the reason mentioned in the spreadsheet ("Not required for the Virtual Prototype project") in the ***Exemption Info*** field.



    iv. Likewise select the other bitfields of the ITCSR register, and select *Untested (Unsafe)*as ***Coverage Exemption*** for them.

    v. Also, select the register ITDR and ITADR and also select *Untested (Unsafe)*as ***Coverage Exemption*** for them.

15. Save the model, re-run the tests, open the Coverage Report, and make sure you hit the Refresh button of the browser to see the latest state of the report. You see that the coverpoints to be reviewed has gone down from 7 to 2. The remaining 2 are interface cover points:

timer.tlmc | Functional coverage report

file:///remote/de02sls02/keding/PA/Training/TLMC/GIT/data/module_1/start_Module1LabB/timer/Tests/Unitt

Result	Failed		
# Coverpoints to be reviewed	2	Reviewed Coverage:	50%
# Coverpoints to be tested	7	Tested Coverage:	50%
Exempt # Coverpoints	3		
Total # Coverpoints	14		

Unreviewed, uncovered coverpoint	Coverage
p_clk	0%
p_irq	0%

Memory Exempted Coverpoints	Untested Unsafe
r_registers.ITCSR.OVW	
r_registers.ITCSR.ITIE	
r_registers.ITCSR.ITIF	
r_registers.ITCSR.RLD	
r_registers.ITCSR.EN	
r_registers.ITDR	
r_registers.ITADR	

Memory Exempted Coverpoints	Unmodeled Functional
r_registers.ITCSR.STOP	Not required for the Virtual Prototype project
r_registers.ITCSR.DOZE	Not required for the Virtual Prototype project

There is no way to exempt interface coverpoints in the GUI, but in most cases it also highly recommended to test all available interfaces, so we will keep on writing tests to cover these interfaces as well.

16. The clock interface coverage point is special: it does not count the toggling of the clock signal as coverage event, but only a change of the clock period – and if this is done before the covermodel is enabled (in the test setup), this is also not counted.

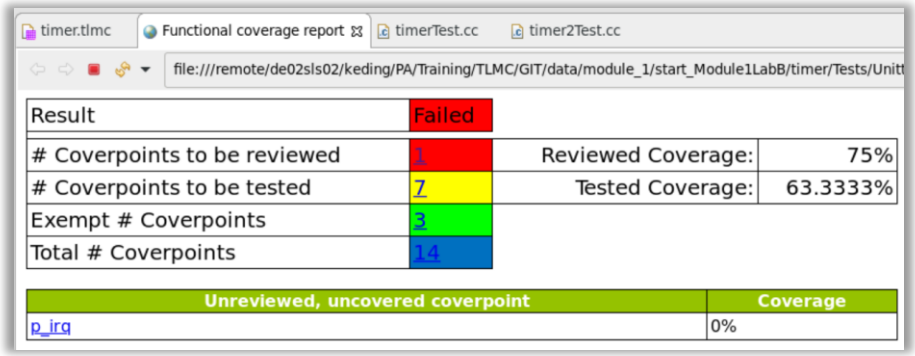
In fact, this coverage point reminds us that the clock period is not set as described in the spec and in the test plan spreadsheet:

		impl	tested	Function Description
Feature				9.6.1 Interval Timer, Mode of Operation
IT_Mode	2			writing to ITDR sets the modulus of timer
IT_Mode	3			reading from ITDR shows modulus
IT_Mode	4			reading from ITADR shows currrent counter value
IT_Mode	5			fixed clock from LOW_REFCLK/4 = 32.768/4 = 8.192 Hz

Hence, you can enhance the setup function in the timerTest.cc file in the following way (the grey code is what should be already there):

```
virtual void setup() {
        // put all pin into default position, do reset
        coverModel->set_enabled(true);
        this->p_clk.set_period_ps(1000000000000/8192); // 8.192 kHZ
        this->m_p_clk_Period = sc_core::sc_time(double(this->p_clk.get_period_ps()), SC_PS);
        wait(2 * this->m_p_clk_Period);
        this->p_reset.set_duration(8*this->m_p_clk_Period);
        this->p_reset.do_reset_pulse();
        wait(2 * this->m_p_clk_Period);
}
```

17. Safe the file, re-run the tests – and you will see that the coverage report now lists the p_clk port as covered, and you also see that both, review and test coverage have improved:



The only uncovered interface is now the interrupt port – you will implement some tests that cover this later, but first there are some more test items (and according to model behavior) from the test plan / spreadsheet to be implemented.

Task-5: Add Model Behavior (TDD)

Motivation: *In the following, we will add some more behavior to the model using the Test Driven Development (TDD) approach.*

18. Add another test for register testing named, `ReadWrite_Test2()` to test reading/writing from/to register using the following information.

    i. Register the new test between `SCML2_BEGIN_TESTS` and `SCML2_END_TESTS` macros in the `timerTest.cc` file.

    ```
    SCML2_TEST(ReadWrite_Test2);
    ```

    ii. Use the `read_write_read32()` function, which first reads a value from the register, then writes the value with all bits flipped, and then reads the register again.

    To be able to use the `read_write_read32()` function, include the `test_utils` header file as follows.

    ```
    #include "test_utils.h"
    ```

    This read-write-read sequence is not exactly the behavior you need for all *timer* registers, though. It could be valid for *ITCSR* register but writing to that register can trigger behavior that changes the other bits as well. It is a status and control register, which means that some bits can be set to trigger behavior, and some can be read only to indicate the status. If the default unit test (or a part for certain register) is not correct, you should first fix it (as is the case for the timer model), and only then you should be adding an implementation.

If you hover your mouse pointer over the function, the C++ editor will pop up the definition of the function:

```
read_write_read32(this->p_registers, this->ITDR, val0, val1);
/* ##################################################################### */
template<class tlmport>
void read_write_read32(tlmport &port, unsigned int address, unsigned int &read1, unsigned int &read2)
{
  bool mbool=false;

  read1 = port.read32(address, &mbool);
  port.write32(address , ~read1);
  read2 = port.read32(address, &mbool);
}
```

The *ITDR* register can be written to specify the modulus of the counter. It is a 16-bit counter so the upper 16 bits are always 0 (refer to the feature no. 7 of *ITDR* in the tracker). Thus, the upper 16 bit of val0 x-ored with valo1 also should be 0.

The *ITADR* register returns the timer counter value and is a read-only register (cannot be written via memory socket), so writing to it should not change the register value and the transaction should return false (`error`). On read, upper 16 bits are also masked and are always read as 0.

We can add write/read transactions and modify assertion as follows:

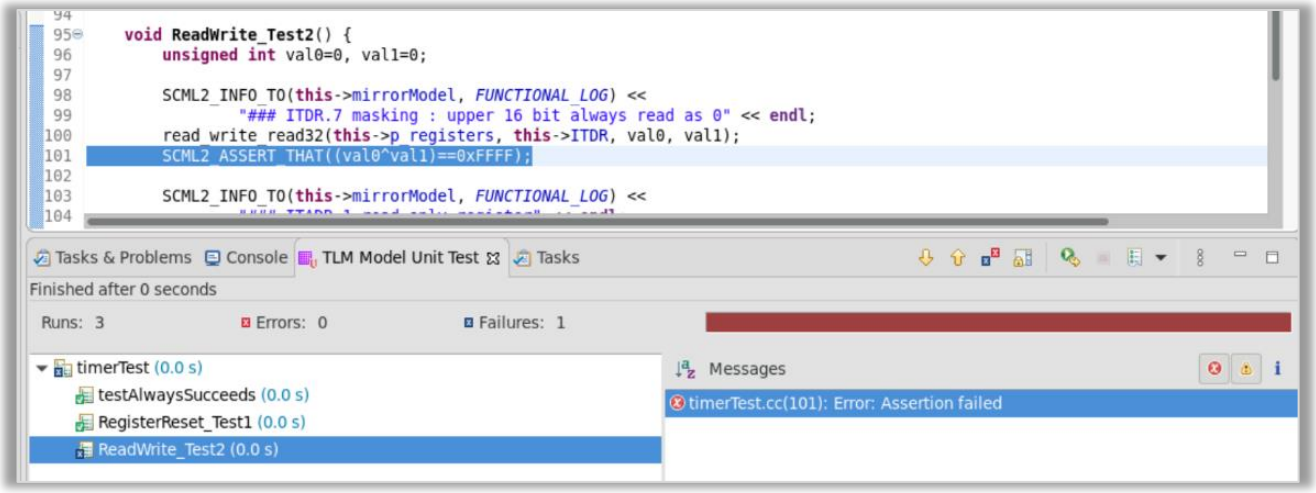
```
void ReadWrite_Test2() {
  unsigned int val0=0, val1=0;

  SCML2_INFO_TO(this->mirrorModel, FUNCTIONAL_LOG) <<
       "### ITDR.7 masking : upper 16 bit always read as 0" << endl;
  read_write_read32(this->p_registers, this->ITDR, val0, val1);
  SCML2_ASSERT_THAT((val0^val1)==0xFFFF);

  SCML2_INFO_TO(this->mirrorModel, FUNCTIONAL_LOG) <<
       "### ITADR.1 read only register" << endl;
  bool mbool = this->p_registers.write32(ITADR, val0);
  SCML2_ASSERT_THAT(mbool==false);

  SCML2_INFO_TO(this->mirrorModel, FUNCTIONAL_LOG) <<
       "### ITADR.4 masking: upper 16bit are always read as zero" << endl;
  val0 = 0;
  val1 = this->p_registers.read32(ITADR, &mbool);
  SCML2_ASSERT_THAT(mbool==true);
  SCML2_ASSERT_THAT((val1>>16)==0x0);
}
```

iii. Save and run the unit tests.



Note: The test will still fail as we have yet not implemented the correct model behavior.

19. Next, let's start to implement the model behavior to have the test passing. The simplest way of always masking out the upper 16bit is to implement an access restriction on the *ITDR* register.

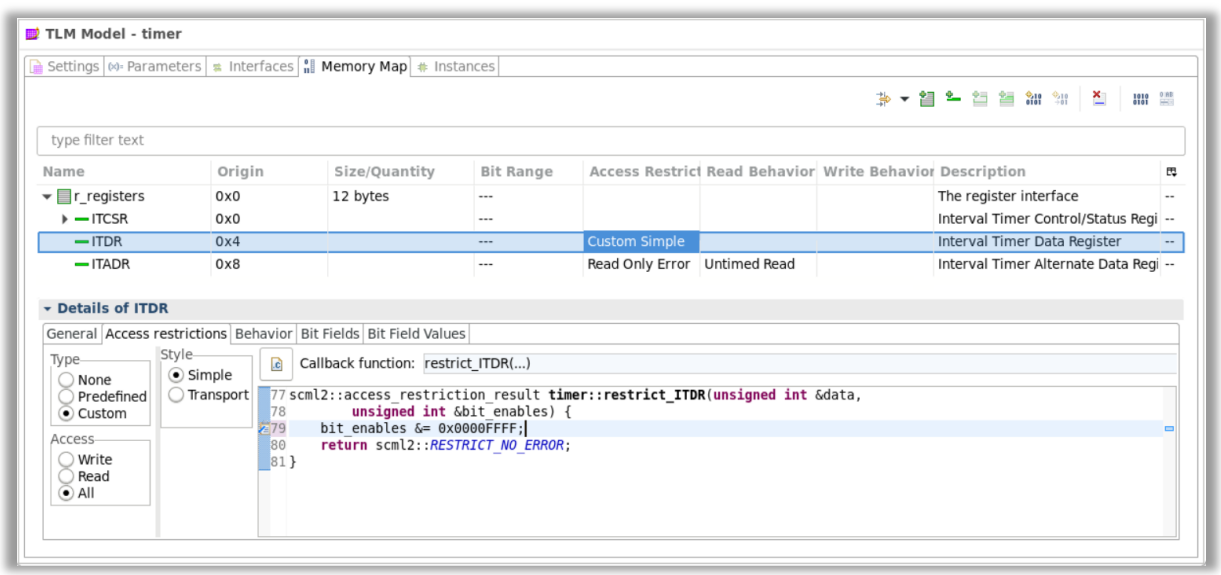
    i. In the ***Memory Map*** page of the `timer.tlmc` project, double-click on the ***Access Restriction*** field of the *ITDR* register and select `Custom Simple` as access restriction type. This means that this access restriction applies both to read and write accesses. Access restrictions are always executed before the behavior callbacks.

    ii. Save the project to generate the default implementation of the access restriction callback.

    iii. In the ***Details of ITDR*** section, select the ***Access Restrictions*** tab. In the peep-hole editor for the callback, replace the `//FIXME…` comment with the following line:

    ```
    bit_enables &= 0x0000FFFF;
    ```

    This only enables the lower 16bit for any following read/write access.



20. Save, re-build, and re-run the unit test. The unit test should be passing now. Mark it as *Implemented and tested* in the feature tracking sheet.

21. Let's now look at the feature list for the next test to be added. We need to test 32-bit accesses to the *ITADR* register of the model (Refer to the Feature 2 for the register in the Tracker).

38	**ITDR**					**9.6.6 ITDR**
39	ITDR		**1**			write the timer modulus
40	ITDR		**2**			value is used during timer reload
41	ITDR		**3**			read return the modulus
42	ITDR		**4**			write can also change counter if OVW is 1
43	ITDR		**5**			counter is read from the ITADR register
44	ITDR		**6**			32 bit access only
45	ITDR		**7**	x	x	masking : upper 16 bit always read as 0
46	ITDR		**8**	x	x	reset: lower 16-bit as 1 (0xFFFF)
47	**ITADR**					**9.6.7 ITADR**
48	ITADR		**1**	x	x	read only register
49	ITADR		**2**			32 bit access only
50	ITADR		**3**			read return the counter value (16 bit)
51	ITADR		**4**	x	x	masking: upper 16bit are always read as zero
52	ITADR		**5**	x	x	reset: lower 16-bit as 1 (0xFFFF)

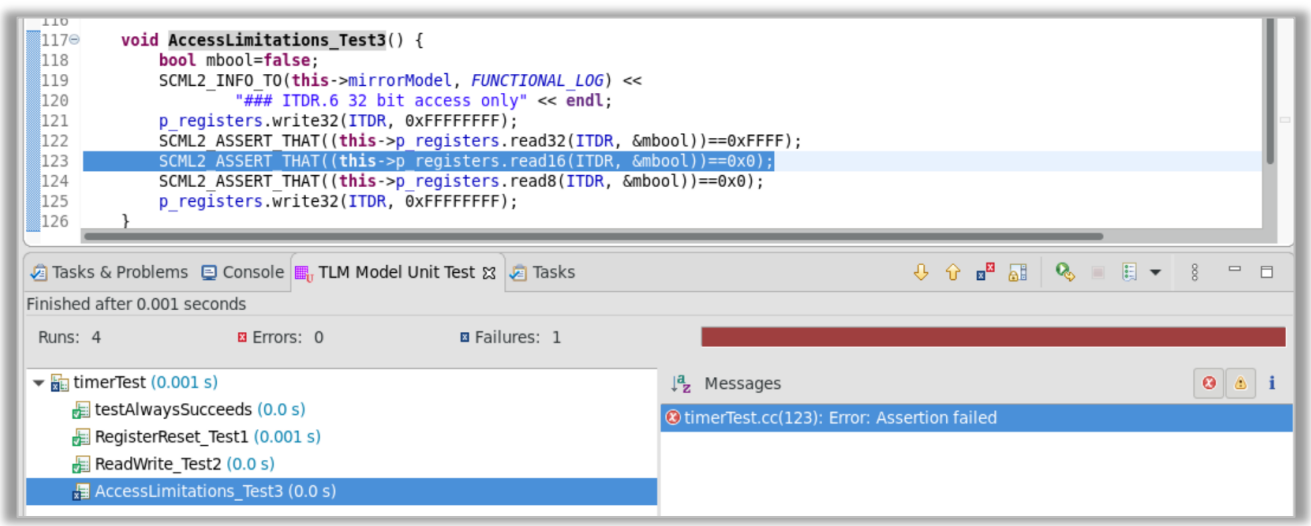
22. For this, lets add a unit test as `AccessLimitations_Test3()`.

    Register the new test between `SCML2_BEGIN_TESTS` and `SCML2_END_TESTS` macros in the `timerTest.cc` file.

23. The implementation code for the `AccessLimitations_Test3` test:

```
void AccessLimitations_Test3() {
        bool mbool=false;
        SCML2_INFO_TO(this->mirrorModel, FUNCTIONAL_LOG) <<
          "### ITDR.6 32 bit access only" << endl;
        p_registers.write32(ITDR, 0xFFFFFFFF);
        SCML2_ASSERT_THAT((this->p_registers.read32(ITDR, &mbool))==0xFFFF);
        SCML2_ASSERT_THAT((this->p_registers.read16(ITDR, &mbool))==0x0);
        SCML2_ASSERT_THAT((this->p_registers.read8(ITDR, &mbool))==0x0);
        p_registers.write32(ITDR, 0xFFFFFFFF);
}
```

Running this test will fail of course, since the according behavior of the timer model is not implemented yet.



24. Let's first implement the access restriction to 32-bit for the *ITDR* register. This register already has one `Custom Simple` access restriction, where we set the `bit_enables` argument to `0x0000FFFF` to mask out the upper 16 bit for the following read/write accesses. Now we'll check in addition if the incoming `bit_enables` mask is different from `0xFFFFFFFF` (i.e. 32-bit access) – if so we issue a warning and disable the data access completely:

```
scml2::access_restriction_result timer::restrict_ITDR(unsigned int& data,
                                                      unsigned int& bit_enables) {
  if ( bit_enables != 0xFFFFFFFF ) {
    SCML2_INFO(FUNCTIONAL_LOG) << "only 32bit access allowed - disabling access (bit_enables= "
                               << bit_enables << ")" << endl;
    bit_enables = 0x0;
  } else {
    bit_enables &= 0x0000FFFF;
  }
  return scml2::RESTRICT_NO_ERROR;
}
```

If you now rebuild your project and run the tests you should see that the assertions on the *ITDR* register are all passing.

Task-6: Adding a Unit test for the Set-and-forget behavior (Optional task)

Motivation: *In the next lab, we will implement the 'set-and-forget' behavior of the timer as described in the functionality tracker (section 9.6.2 Set-and-forget). The starting point for the next lab includes a unit test for the behavior*
Instead of using the unit test in the starting point of Module 1 Lab C, you can write your own unit test in this task. We will provide you with few suggestions and hints.

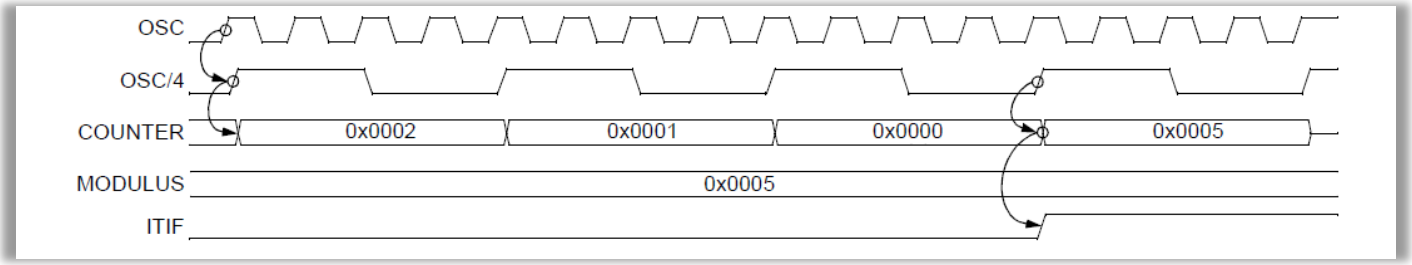
25. Create and register a new unit test `SetAndForget_Test4`. We recommend you write unit tests incrementally by stimulating the model and then checking the results using assertions. Check after each generated stimuli for the changes, do not wait for the end of the test routine to check all the results. An incremental approach eases debugging. So, the recommended coding cycle is:

    i. Write stimuli to the model by using its pins and TLM/memory ports.

    ii. Allow the model to execute behavior by giving control to the SystemC kernel, by calling `wait()`.

    iii. Check the pin or register status (through the TLM port).

The behavior to be tested and also the associated timing becomes quite clear from the charts in the PIT spec:



26. The unit test should first write a modulus value (lets say:. `0x5`) to the *ITDR* register.

27. Define the bit values needed for comparison, after the `include` statements as follows.

```
#include <timerTestHarness.h>

#include "test_utils.h"

// Define the bit values

#define ENbit   0x1

#define RLDbit  0x2

#define ITIFbit 0x4

#define ITIEbit 0x8

#define OVWbit  0x10
```

28. Next you need to write to the *ITCSR* register to set the following bits to `1`:

    i. *RLD* (SF.1: SF operation mode is selected when RLD is 1)

    ii. *ITIE* (SF.4: if the ITIE is set the interrupt pin is fired on timer expiration)

    iii. *EN* (ITCSR.12: enable the PIT)

29. Assert that after one cycle

    i. The *ITCSR* registers still has only the *RLD*, *ITIE*, and *EN* bits set.

    ii. The *ITDR* register still carries the modulus value (`0x5`)

    iii. The *ITADR* register, representing the current counter value, carries the modulus value reduced by 1 (`0x4`)

    iv. The interrupt *p_irq* is not enabled yet.

30. Assert that after two cycles the value of the *ITADR* register is reduced by two (`0x3`).

31. Assert that after 5 cycles the value of the *ITADR* register is reduced by 5 (`0x0`).

32. Assert that after 6 cycles

    i. The value of the *ITADR* register is reloaded with the modulus value again (0x5)

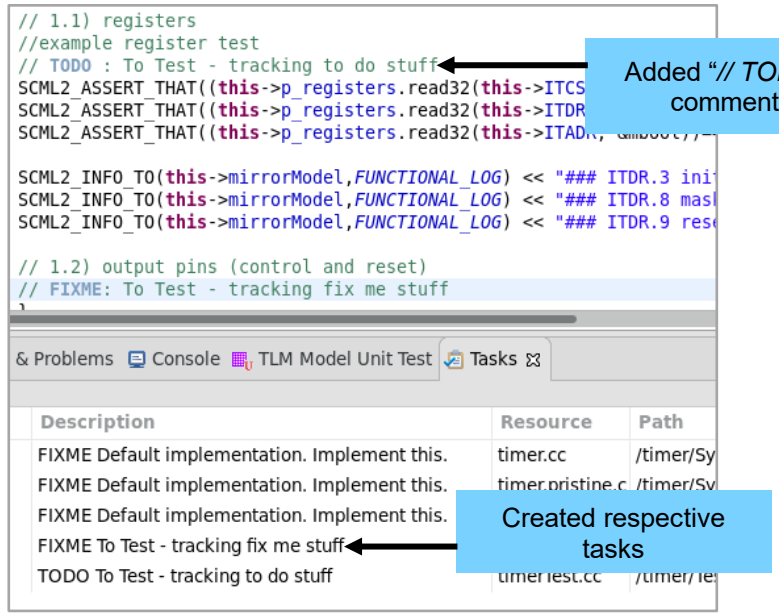
    ii. The *ITCSR* registers has the *RLD*, *ITIE*, *EN*, and *ITIF* bits set.

iii. The interrupt *p_irq* is enabled.

33. Run the test – it should of course fail at this state, as the according model behavior is not implemented yet. Keep repeating the steps till you complete your test.

Tip: To keep track of what items are still pending for testing and implementation you can prepare a unit test with a list of features to be tested such as missing *ITDR* and *ITADR* features. Add "*TODO*" or "*FIXME*" keywords to your code comment.

This will automatically create tasks in the ***Tasks*** tab. You can then easily track what still needs to be done directly in the Virtualizer Studio.



To open this tab, click ***Window → Show View → Other -> General → Tasks***.

SetAndForget_Test4() code for your reference

```
#include <timerTestHarness.h>
#include "test_utils.h"
// Define the bit values
#define ENbit   0x1
#define RLDbit  0x2
#define ITIFbit 0x4
#define ITIEbit 0x8
#define OVWbit  0x10

void SetAndForget_Test4() {
        SCML2_INFO_TO(this->mirrorModel, FUNCTIONAL_LOG) <<
            "### IT_Mode.2: writing to ITDR sets the modulus of timer" << endl;
        p_registers.write32(ITDR, 0x5);
        wait(m_p_clk_Period);
        SCML2_INFO_TO(this->mirrorModel, FUNCTIONAL_LOG) <<
            "### SF.1: SF operation mode is selected when RLD is 1" << endl;
        SCML2_INFO_TO(this->mirrorModel, FUNCTIONAL_LOG) <<
            "### SF.4: if the ITIE is set then interrupt pin is fired" << endl;
        SCML2_INFO_TO(this->mirrorModel, FUNCTIONAL_LOG) <<
            "### ITCSR.13: enable the PIT" << endl;
        unsigned int cs = ENbit|RLDbit|ITIEbit;
        p_registers.write32(ITCSR, cs);
        wait(m_p_clk_Period); // 1 cycle after start
        SCML2_ASSERT_THAT(this->p_registers.read32(ITDR).equals(0x5));
        SCML2_ASSERT_THAT(this->p_registers.read32(ITADR).equals(0x4));
        SCML2_ASSERT_THAT(this->p_registers.read32(ITCSR).equals(cs));
        SCML2_ASSERT_THAT(this->p_irq.get_enabled(false));
        unsigned int p_clk_Period_ps = this->m_p_clk_Period.value_in_ps();
        unsigned int p_clk_Period_ns = (p_clk_Period_ps+999)/1000 ;
        SCML2_INFO_TO(this->mirrorModel, FUNCTIONAL_LOG) <<
            "One clock cycle is " << p_clk_Period_ps << "ps = " <<
            p_clk_Period_ns << " ns" << endl;
        wait(m_p_clk_Period); // 2 cycles after start
        SCML2_ASSERT_THAT(this->p_registers.read32(ITADR).equals(0x3));
        SCML2_INFO_TO(this->mirrorModel, FUNCTIONAL_LOG) <<
            "### SF.2: after counted to 0x0 the ITIF flag is set" << endl;
```

```
        SCML2_INFO_TO(this->mirrorModel, FUNCTIONAL_LOG) <<
            "### SF.3: after counter reached 0x0 the modulus is loaded back into counter" <<
endl;

        SCML2_ASSERT_THAT(this-
>p_registers.read32(ITADR).equals(0x0).after_ns(3*p_clk_Period_ns));
        wait(m_p_clk_Period); // 6 cycles after start
        SCML2_ASSERT_THAT(this->p_registers.read32(ITADR).equals(0x5));
        SCML2_ASSERT_THAT(this->p_registers.read32(ITCSR).equals(cs|ITIFbit));
        SCML2_ASSERT_THAT(this->p_irq.get_enabled(true));
}
```

Task-7: What's Next

In the next lab we will implement the `set-and-forget` behavior of the timer.